

Actividad final integradora: Devoluciones sin correo en Mercado Austral

PF1821 · Construcción de Agentes y Automatización con Herramientas Low Code
Módulo 2 · MA04560 · Fundamentos de n8n y manipulación de datos

MOMENTO	MODALIDAD	CALIFICACIÓN
Al cierre del módulo, después del último ABPRO	Individual, con entrega en el LMS	Pauta de 40 puntos por componentes de la competencia, exigencia 60 %

COMPETENCIA DEL MÓDULO

Crear workflows básicos de automatización utilizando n8n, manipulando datos mediante nodos fundamentales e integrando bases de datos básicas, para resolver problemáticas empresariales de acuerdo con buenas prácticas de automatización.

CONTENIDOS DEL PLAN

Los cuatro contenidos del módulo, todos integrados: 1 · Conceptos fundamentales de automatización de workflows y las características de n8n; 2 · Creación de workflows en la interfaz de usuario de n8n; 3 · Nodos core de n8n y manipulación de datos; 4 · Lógica condicional y debugging en n8n

Contexto

Mercado Austral (empresa ficticia) es la distribuidora de abarrotes que te acompañó todo el módulo: vende a clientes minoristas y a almacenes mayoristas por un formulario web y ya guarda sus pedidos en la tabla `pedidos` de Supabase. Con los pedidos en orden, el problema se movió al final del proceso: las devoluciones.

Hoy, el cliente que quiere devolver algo escribe un correo. Una asistente de atención al cliente lo lee, busca el pedido, revisa que el correo sea del mismo cliente, anota la devolución en una planilla y avisa por correo al área que corresponde: **calidad** si el producto llegó dañado, **bodega** si hubo un error de despacho y **atención al cliente** si la persona se arrepintió. Los pedidos grandes y los de clientes mayoristas los aprueba la jefatura antes de devolver el dinero. Cada viernes, la asistente arma a mano un resumen para finanzas y una lista de retiros para **Transportes Cordillera** (empresa ficticia), que pasa a buscar los productos al domicilio del cliente.

El proceso funciona, pero es lento y se equivoca. Llegan unas 60 devoluciones al mes, 1 de cada 10 queda mal registrada (con el pedido o el motivo equivocado) y los clientes esperan días una respuesta. Nadie sabe cuántas devoluciones hay por motivo hasta el viernes, ni cuáles siguen esperando retiro.

El desafío

Mercado Austral quiere dejar el correo atrás. Te pide una propuesta con números y dos workflows en n8n conectados a su Supabase:

- **Registro de devoluciones.** El cliente llena un formulario; el workflow valida la solicitud contra el pedido, decide si requiere aprobación, la envía al área que corresponde, la registra y le responde al cliente con su número de devolución. Lo que no calce va a revisión manual: nada se pierde en silencio.
- **Cierre semanal de devoluciones.** Cada viernes, un resumen por motivo en CSV para finanzas y un archivo XML con los retiros para Transportes Cordillera; el lunes, la lectura de la confirmación del transportista para marcar los retiros programados.

Todo tiene que quedar probado y trazable: para cualquier devolución, alguien de la empresa debe poder saber por qué ruta pasó y en qué ejecución se procesó.

Lo que vas a construir

Parte 1 · La propuesta

1. **El costo del proceso actual.** Con los datos del proceso (en Insumos), calcula las horas de la asistente y el costo mensual de gestionar las devoluciones a mano: la gestión de cada devolución, las correcciones y el cierre del viernes. Muestra cada cálculo.
2. **Retorno de la inversión.** Calcula el ahorro mensual neto (horas liberadas, descontando lo que seguirá atendiendo una persona, menos la herramienta y la mantención), el ROI a 12 meses con la fórmula $ROI = (\text{beneficio} - \text{costo}) \div \text{costo}$, con la construcción, la herramienta y la mantención dentro del costo, y el plazo en que se recupera la inversión.
3. **Por qué automatizarlo y tres beneficios.** Explica por qué este proceso conviene automatizarlo según las cuatro condiciones (repetitivo, frecuente, con reglas claras y con datos digitales) y qué parte seguirá en manos de una persona (la aprobación de la jefatura y la revisión manual). Descríbelo como workflow: qué lo inicia, qué pasos tiene y qué resultado produce. Nombra tres beneficios para la empresa o sus clientes, al menos uno que no se mida en dinero, e indica a qué caso de uso empresarial corresponde este proceso: el área de la empresa y el tipo de automatización.
4. **Por qué n8n.** Compara n8n con Zapier y Make en tres criterios que importan en este caso: dónde quedan los datos de los clientes (autoalojamiento), qué unidad cuenta el plan de pago de cada una y cuánta lógica propia admite (expresiones y nodo Code). Verifica los precios y planes vigentes en el sitio de cada proveedor y cita la fuente con la fecha de consulta. Cierra con una recomendación de 3 a 5 líneas.
5. **Diagrama de la solución.** Dibuja los dos workflows con sus nodos y conexiones, las credenciales que usan, las tres tablas de Supabase con su relación y dónde se revisan las ejecuciones. Puede ser digital o a mano, siempre que se lea bien.
6. **Lista de nodos.** Enumera los nodos que vas a usar, clasificados por tipo: triggers, nodos core (transformación y lógica) e integraciones con aplicaciones. Marca el que conecta con un servicio que no tiene nodo propio en n8n.

Parte 2 · Workflow "Registro de devoluciones"

1. **Tablas.** Ejecuta en el editor SQL de Supabase el script de Insumos: agrega la columna `estado` a `pedidos`, carga los seis pedidos de prueba y crea `devoluciones` y `log_devoluciones`. Crea o reutiliza en n8n la credencial de tu proyecto de Supabase.
2. **Formulario.** Crea el workflow con un *n8n Form Trigger* titulado "Solicitud de devolución" y cuatro campos con estas etiquetas exactas: `pedido_id` (Number, obligatorio), `email` (Email, obligatorio), `motivo` (lista desplegable con producto dañado, error de despacho y arrepentimiento; obligatorio) y `comentario` (texto largo, opcional).
3. **Configuración.** Los valores que pueden cambiar no se escriben dentro de los nodos: el umbral de aprobación (50000), el monto mínimo para retirar el producto (10000) y los nombres de las tablas `devoluciones` y `log_devoluciones`. Léelos como variables: `$env.UMBRAL_APROBACION` en una instancia propia que lo permita, o `$vars.UMBRAL_APROBACION` si tu plan de n8n tiene *Variables*. Si tu instancia no permite ninguna de las dos, crea al comienzo un *Edit Fields* llamado "Configuración" con esos cuatro valores, con *Include Other Input Fields* activado para que los campos del formulario sigan hasta "Normalizar devolución", y explica en el informe cómo pasarían a variables de entorno. Las variables llegan como texto: conviértelas con `Number()` antes de comparar.
4. **Normalización.** Un *Edit Fields (Set)* llamado "Normalizar devolución" declara el tipo de cada campo: `pedido_id` (Number), `email` sin espacios en los extremos y en minúsculas, `motivo` sin espacios en los extremos y en minúsculas (texto vacío si no llega), `comentario` sin espacios en los extremos y `recibido_en` (Date & Time) con la fecha y hora de la ejecución.
5. **Buscar el pedido.** Nodo Supabase "Buscar pedido", operación *Get a row*, en `pedidos`, con `id` igual a `{{ $json.pedido_id }}`. En los *Settings* del nodo activa *Always Output Data*: si el pedido no existe, el nodo entrega un ítem vacío en vez de ninguno y el paso siguiente lo puede detectar. Sin esa opción, la solicitud desaparece sin error.
6. **Casos borde.** Dos nodos *If* separan lo que no se puede registrar: "¿Existe el pedido?" (el `id` del pedido existe) y "¿Coincide el correo?" (el correo del pedido, normalizado, es igual al del formulario, que lees con `$('Normalizar`

devolución').item.json.email). Cada salida falsa va a la rama de revisión manual con un **detalle** que diga por qué: "pedido inexistente" o "correo no coincide".

7. **Armar la devolución.** Un *Edit Fields* "Armar devolución" junta los datos del formulario con dos datos del pedido, **total_pedido** (Number) y **comuna** (String), y calcula dos campos Boolean con operadores: **requiere_aprobacion**, verdadero si el total es mayor que el umbral o el cliente es mayorista, y **requiere_retiro**, verdadero si el total es mayor o igual que el monto mínimo de retiro. Con el nodo "Configuración", la primera queda así: `{{ $json.total > $('Configuración').item.json.umbral_aprobacion || $json.tipo_cliente === 'mayorista' }}`.
8. **Enrutar por motivo.** Un *Switch* en modo *Rules* con tres salidas con nombre: "producto dañado" a calidad, "error de despacho" a bodega y "arrepentimiento" a atención al cliente. Activa *Fallback Output* con *Extra Output* y conéctala a la revisión manual con el detalle "motivo fuera de la lista". En cada una de las tres salidas, un *Edit Fields* con *Include Other Input Fields* activado escribe el **area**: **calidad**, **bodega** o **atencion_cliente**.
9. **Registrar.** Nodo Supabase "Crear devolución", *Create a row* en **devoluciones** (con **pedido_id** como clave foránea). En *Data to Send*, elige *Define Below for Each Column* y asigna **pedido_id**, **email**, **motivo**, **area**, **comentario**, **total_pedido**, **comuna**, **requiere_aprobacion**, **requiere_retiro** y **recibido_en**; **id**, **creado_en**, **estado** y **fecha_retiro** los pone la base de datos. A continuación, "Marcar pedido", *Update a row* en **pedidos**, que deja ese pedido con **estado** = **devolucion_en_curso**. En los *Settings* de "Crear devolución", usa *On Error* con *Continue (using error output)* y lleva esa salida al registro, con la ruta **error** y el mensaje del error.
10. **Dejar rastro.** Toda ejecución termina con una fila en **log_devoluciones**, también las que van a revisión manual: **id_ejecucion** = `{{ $execution.id }}`, **workflow** = `{{ $workflow.name }}`, **pedido_id**, **devolucion_id** (vacío si no se creó), **ruta** y **detalle**.
11. **Responder al cliente.** Un *Form Ending* muestra el número de devolución (`{{ $('Crear devolución').item.json.id }}`) y, si requiere aprobación, avisa que la jefatura la revisará antes de devolver el dinero. La rama de revisión manual termina en otro *Form Ending* que avisa que una persona revisará la solicitud y escribirá al correo indicado, sin mostrar ningún dato del pedido: quien escribe podría no ser quien hizo el pedido.

Parte 3 · Workflow "Cierre semanal de devoluciones"

1. **Trigger.** Un *Schedule Trigger* los viernes a las 17:00. Revisa en los ajustes del workflow que la zona horaria sea la de Chile (America/Santiago). Mientras construyes, ejecútalo desde el editor.
2. **Las devoluciones de la semana.** Nodo Supabase, *Get many rows* en **devoluciones**, con *Return All* activado y el filtro **creado_en** mayor o igual que `{{ $now.minus({ days: 7 }).toISOString() }}`.
3. **Resumen para finanzas.** Un *Summarize* agrupa por **motivo** y calcula *Count* de **id** y *Sum* de **total_pedido**. Un *Edit Fields* deja los nombres que pide finanzas (**motivo**, **devoluciones** y **monto_total**) y *Convert to File*, con *Convert to CSV*, genera **devoluciones_semana.csv**. Antes de ejecutar, calcula a mano qué debería entregar el resumen con tus devoluciones de prueba y compáralo con el resultado.
4. **Lo que hay que retirar.** En una rama paralela, que sale del mismo *Schedule Trigger* del viernes, otro *Get many rows* en **devoluciones**, con *Return All* activado, sin filtro de fecha y con el filtro **estado** igual a **registrada**, trae todas las que siguen sin retiro programado: así se vuelven a pedir los retiros pendientes de semanas anteriores y no se piden de nuevo los ya programados. Después, un *Filter* deja las que tienen **requiere_retiro** verdadero.
5. **Completar con el pedido.** Un tercer *Get many rows*, con *Return All* activado, trae los pedidos. Conéctalo directamente al *Schedule Trigger* del viernes, que entrega un solo ítem, para que se ejecute una vez (si lo pones después de otro nodo, se ejecuta una vez por cada ítem que recibe: activa *Execute Once* en sus *Settings*). Un *Merge* en modo *Combine*, por *Matching Fields*, une cada devolución que sale del *Filter* (entrada 1, **pedido_id**) con su pedido (entrada 2, **id**) para agregar **nombre** y **producto**. Las dos tablas tienen campos con el mismo nombre (**id**, **creado_en**, **email**, **comuna** y **estado**): en las opciones del *Merge*, decide cómo resolver ese choque (*Clash Handling*) y comprueba en la salida que el **id** y el **estado** sean los de la devolución. Cuenta los ítems: deben salir tantos como devoluciones entraron.
6. **El archivo del transportista.** Un *Edit Fields* deja los campos que pide Transportes Cordillera (formato en Insumos). Como el nodo *XML* convierte cada ítem por separado, primero junta todos los retiros en un solo ítem con *Aggregate (All Item Data)*, en el campo **retiro**. Luego, el nodo *XML* en modo *JSON to XML*, con **retiros** como nombre raíz (*Root Name*), y *Convert to File*, con *Convert to Text File*, para guardar **retiros_semana.xml**.

- 7. La confirmación del transportista.** Transportes Cordillera responde con un archivo JSON (en Insumos, y publicado en el LMS como `confirmacion_retiros.json`). Agrega al mismo workflow un segundo *Schedule Trigger*, los lunes a las 9:00, y lee la confirmación con *HTTP Request* (GET al enlace publicado, respuesta en JSON). Si tu instancia no llega al enlace, fija el JSON como datos del nodo.
- 8. Un ítem por retiro.** Un *Split Out* sobre el campo `retiros` convierte el array en un ítem por retiro. Un *If* separa los que tienen `estado` igual a `programado` de los que no.
- 9. Actualizar.** Para los programados, nodo Supabase *Update a row* en `devoluciones`, con dos condiciones, `pedido_id` igual al del retiro y `estado` igual a `registrada`, que deja `estado` = `retiro_programado` y la `fecha_retiro`. Los que no se programaron siguen en `registrada` y vuelven a salir el viernes siguiente. Registra cada decisión en `log_devoluciones`, con la ruta `retiro_programado` o `sin_cobertura` y el `id_ejecucion`.

Parte 4 · Pruebas, depuración y trazabilidad

- 1. Plan de pruebas.** Antes de ejecutar, completa una tabla con las ocho devoluciones de prueba de Insumos (puedes agregar casos propios) y el resultado que esperas de cada una. Así se ve la primera fila:

Caso	Ruta esperada	<code>requiere_aprobacion</code>	<code>requiere_retiro</code>	Filas esperadas	Mensaje final	Resultado obtenido	<code>id_ejecucion</code>
D1	calidad	false	true	1 en devoluciones y 1 en <code>log_devoluciones</code>	Número de devolución		

- 2. Ejecución.** Envía los casos D1 a D7 por la URL de prueba del formulario, uno por ejecución, como ocurrirá en producción. La D8 no se puede elegir en la lista del formulario: pruébala fijando los datos del trigger (*pin data*) y editando el `motivo`. Anota el resultado obtenido y el `id_ejecucion` de cada caso. Después de cada corrección, vuelve a correr todos los casos, no solo el que falló.
- 3. Evidencia.** En el historial de ejecuciones, filtra por workflow y captura la lista con tus ejecuciones de prueba. Para al menos un caso de revisión manual, abre su ejecución y captura cuántos ítems entraron y salieron del nodo que lo desvió.
- 4. Cierre semanal.** Ejecuta el cierre con las devoluciones que crearon tus pruebas y verifica el CSV, el XML (un `<retiro>` por cada devolución que requiere retiro) y el estado de las devoluciones después de leer la confirmación. Si tus devoluciones de prueba tienen más de 7 días, el resumen para finanzas sale vacío: borra las filas de prueba (primero las de `log_devoluciones`) y vuelve a enviar los casos antes de ejecutar el cierre.
- 5. Nota de depuración.** Describe al menos una falla que encontraste mientras construías: el síntoma, cómo la detectaste (qué herramienta de n8n usaste), la causa, la corrección y la prueba que confirma que quedó resuelta.
- 6. Orden del workspace.** Nombra los workflows "MA · Registro de devoluciones" y "MA · Cierre semanal de devoluciones", agrégalas las etiquetas `mercado-austral` y `devoluciones`, renombra cada nodo por lo que hace ("Buscar pedido", no "Supabase1") y deja una nota en el canvas que explique qué motivo va a cada salida del Switch, por qué sus reglas no se pisan, a dónde va lo que no calza (salida de respaldo) y de dónde sale la configuración.

Insumos

Datos del proceso actual

Son supuestos del caso ficticio, para calcular el costo y el retorno de la Parte 1.

Dato	Valor
Devoluciones al mes	60
Gestión manual de cada devolución (leer el correo, buscar el pedido, registrar y avisar al área)	15 minutos de la asistente
Devoluciones mal registradas	1 de cada 10
Corrección de cada devolución mal registrada	20 minutos más de la asistente

Dato	Valor
Cierre del viernes (resumen para finanzas y lista de retiros)	1,5 horas de la asistente por semana; el mes tiene 4 semanas
Costo de una hora de la asistente para la empresa	\$7.000
Devoluciones que seguirán necesitando a una persona después de automatizar (revisión manual o aprobación)	12 al mes, con 10 minutos de la asistente cada una
Construcción, pruebas y puesta en producción de la solución por una persona técnica externa, una sola vez	\$360.000
Mantenición de la solución	2 horas-persona al mes de la misma persona técnica, a \$12.000 la hora
Herramientas: la parte del plan de n8n y de Supabase que se asigna a este proceso	\$35.000 al mes

Script para Supabase

Ejecútalo en el editor SQL de tu proyecto. Si tu tabla `pedidos` ya tiene más columnas (por ejemplo, las que agregaste en los ABPRO), el script funciona igual mientras esas columnas acepten valores nulos o tengan un valor por defecto.

```

-- 1. La tabla pedidos del curso (solo se crea si tu proyecto aún no la tiene)
create table if not exists pedidos (
  id                bigint generated always as identity primary key,
  creado_en         timestampz not null default now(),
  nombre            text not null,
  email             text not null,
  comuna            text not null,
  tipo_cliente      text not null check (tipo_cliente in ('minorista', 'mayorista')),
  producto          text not null,
  cantidad          integer not null check (cantidad > 0),
  precio_unitario   integer not null,
  total            integer not null
);

-- 2. Estado del pedido, que el registro de devoluciones actualiza
alter table pedidos add column if not exists estado text not null default 'registrado';

-- 3. Seis pedidos de prueba con id fijo (901 a 906)
insert into pedidos (id, creado_en, nombre, email, comuna, tipo_cliente, producto, cantidad,
precio_unitario, total)
overriding system value
values
  (901, '2026-10-05 10:12:00-03', 'Ana Rojas',          'ana.rojas@correo.test',  'Ñuñoa',
'minorista', 'Café en grano 1 kg', 2, 12500, 25000),
  (902, '2026-10-05 16:40:00-03', 'Minimarket Doña Rosa', 'rosa@donarosa.test',    'Maipú',
'mayorista', 'Arroz 25 kg',      8, 24500, 196000),
  (903, '2026-10-06 09:05:00-03', 'Pedro Soto',          'pedro.soto@correo.test',  'Maipú',
'minorista', 'Té verde 100 u',   2, 4200, 8400),
  (904, '2026-10-06 11:30:00-03', 'Almacén El Sol',      'compras@elsol.test',     'Temuco',
'mayorista', 'Azúcar 25 kg',     2, 21900, 43800),
  (905, '2026-10-07 13:15:00-03', 'Camila Pérez',        'camila.perez@correo.test', 'Valparaíso',
'minorista', 'Aceite 5 L',       4, 15990, 63960),
  (906, '2026-10-07 18:20:00-03', 'Jorge Muñoz',         'jorge.munoz@correo.test',  'Ñuñoa',
'minorista', 'Yerba mate 1 kg',  3, 6900, 20700)
on conflict (id) do nothing;

-- Para que los próximos pedidos sigan numerándose después del mayor id cargado
select setval(pg_get_serial_sequence('pedidos', 'id'), (select max(id) from pedidos));

-- 4. Devoluciones: cada una apunta a su pedido
create table devoluciones (
  id                bigint generated always as identity primary key,
  creado_en         timestampz not null default now(),
  pedido_id         bigint not null references pedidos (id),
  email            text not null,
  motivo           text not null check (motivo in ('producto dañado', 'error de despacho',
'arrepentimiento')),
  area             text not null check (area in ('calidad', 'bodega', 'atencion_cliente')),
  comentario        text,
  total_pedido     integer not null, -- foto del pedido al momento de la devolución:
  comuna           text not null,    -- el monto que se evaluó y la comuna del retiro
  requiere_aprobacion boolean not null,
  requiere_retiro   boolean not null,
  recibido_en       timestampz,      -- cuándo llegó la solicitud (lo calcula el workflow)
  estado           text not null default 'registrada'
  check (estado in ('registrada', 'retiro_programado', 'cerrada')),
  fecha_retiro      date
);

-- 5. Registro de cada ejecución, con su ruta
create table log_devoluciones (
  id                bigint generated always as identity primary key,
  creado_en         timestampz not null default now(),
  id_ejecucion      text not null,
  workflow          text not null,
  pedido_id         bigint, -- sin clave foránea: también se registran pedidos que no existen
  devolucion_id     bigint references devoluciones (id),
  ruta             text not null,

```

```
detalle      text
);
```

`total_pedido` y `comuna` se copian a propósito: son la foto del pedido que justificó la decisión (el monto que se comparó con el umbral y la comuna donde se retira). El resto de los datos del pedido se obtiene por la clave foránea. Para repetir las pruebas desde cero, borra primero las filas de `log_devoluciones` y después las de `devoluciones`: la clave foránea no deja borrar una devolución que el registro todavía apunta. Los pedidos no se borran.

Devoluciones de prueba

Caso	pedido_id	email (como lo escribe el cliente)	motivo	comentario
D1	901	ana.rojas@correo.test	producto dañado	El paquete de café llegó roto.
D2	906	Jorge.Munoz@Correo.TEST	error de despacho	Pedí yerba mate y llegó té verde.
D3	903	pedro.soto@correo.test	arrepentimiento	Compré de más.
D4	905	camila.perez@correo.test	producto dañado	Dos bidones llegaron abollados y con filtración.
D5	904	compras@elsol.test	arrepentimiento	Nos equivocamos de producto al hacer el pedido.
D6	902	compras@elsol.test	error de despacho	Llegó arroz integral en vez de arroz blanco.
D7	915	ana.rojas@correo.test	producto dañado	El pedido llegó incompleto.
D8	902	rosa@donarosa.test	producto vencido (no está en la lista: se prueba con datos fijados)	Dos sacos venían con la fecha vencida.

Formato del archivo de retiros

Así pide Transportes Cordillera el archivo XML: un elemento `<retiro>` por cada devolución que requiere retiro. El `devolucion_id` es el que asignó tu base de datos.

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<retiros>
  <retiro>
    <devolucion_id>12</devolucion_id>
    <pedido_id>901</pedido_id>
    <nombre>Ana Rojas</nombre>
    <comuna>Ñuñoa</comuna>
    <producto>Café en grano 1 kg</producto>
    <motivo>producto dañado</motivo>
  </retiro>
</retiros>
```

Confirmación del transportista (confirmacion_retiros.json)

```
{
  "transportista": "Transportes Cordillera",
  "archivo": "retiros_semana.xml",
  "respondido_en": "2026-10-19T08:30:00-03:00",
  "retiros": [
    { "pedido_id": 901, "comuna": "Ñuñoa", "estado": "programado", "fecha_retiro": "2026-10-20",
      "franja": "09:00-13:00" },
    { "pedido_id": 906, "comuna": "Ñuñoa", "estado": "programado", "fecha_retiro": "2026-10-20",
      "franja": "14:00-18:00" },
    { "pedido_id": 905, "comuna": "Valparaíso", "estado": "programado", "fecha_retiro": "2026-10-21",
      "franja": "09:00-13:00" },
    { "pedido_id": 904, "comuna": "Temuco", "estado": "sin_cobertura", "fecha_retiro": null,
      "franja": null,
      "observacion": "Sin retiros en Temuco esta semana; volver a solicitar." }
  ]
}
```

Condiciones

- **Es individual.** Puedes consultar las lecturas del módulo, la documentación oficial de n8n y de Supabase y preguntar en el foro, pero sin compartir tu solución.
- **Solo datos ficticios.** Trabaja con los datos de Insumos, con correos de dominio `.test`. No uses datos de personas ni de empresas reales.
- **Ninguna credencial ni clave en la entrega.** Ni la clave de servicio de Supabase ni otra clave o token pueden aparecer en un parámetro, en el nodo "Configuración", en los datos fijados, en el informe o en una captura. El workflow exportado guarda el nombre de la credencial, no la clave, pero los datos fijados sí se exportan: revisa el archivo antes de subirlo.
- **Puedes reutilizar lo que construiste en el módulo,** por ejemplo la credencial de Supabase, la normalización de «Registro de pedidos, versión 1», el cruce con Merge de «Mercado Austral pasa a una base de datos» o los registros de «El semáforo de despachos». Indica en el informe qué reutilizaste y qué cambiaste.
- **Versiones de n8n.** Algunos botones y opciones cambian de nombre entre versiones. Si en la tuya algo se llama distinto, usa el equivalente y anótalo en el informe.

Entrega

Sube a la Tarea del LMS, en una sola entrega:

1. `informe-devoluciones.pdf`, con la propuesta (Parte 1), el plan de pruebas con resultados esperados y obtenidos, la nota de depuración, lo que reutilizaste y las capturas: las tablas `devoluciones` y `log_devoluciones` con las filas de prueba, el historial de ejecuciones, la ejecución de un caso de revisión manual y el canvas de los dos workflows.
2. Los dos workflows exportados en JSON (menú del workflow, *Download*): `registro-devoluciones.json` y `cierre-semanal-devoluciones.json`.
3. Los archivos que generó el cierre semanal: `devoluciones_semana.csv` y `retiros_semana.xml`.

Cómo se evalúa

La pauta tiene una sección por cada componente de la competencia del módulo. Cada fila se asigna completa si se cumple, a la mitad si se cumple en parte y en cero si no está. El tutor/a importa tus workflows en su propia instancia, con sus tablas del insumo, y ejecuta algunos de tus casos de prueba.

Componente de la competencia	Qué se revisa	Puntos
Para resolver problemáticas empresariales	Parte 1	6

Componente de la competencia	Qué se revisa	Puntos
	Costo actual, ahorro neto, ROI a 12 meses y plazo de recuperación, calculados con los datos del caso y con el cálculo a la vista	2
	Por qué el proceso conviene automatizarlo y su descripción como workflow (inicio, pasos y resultado); tres beneficios concretos, uno que no se mide en dinero, y el caso de uso empresarial al que corresponde	1
	Comparación con Zapier y Make en los tres criterios pedidos, con fuentes y fecha de consulta, y una recomendación para el caso	2
	Diagrama con nodos, conexiones, credenciales, tablas y ejecuciones, y lista de nodos clasificada por tipo	1
Crear workflows básicos de automatización utilizando n8n	<i>Partes 2, 3 y 4</i>	8
	Form Trigger con los cuatro campos, sus tipos y su obligatoriedad; dos Form Ending: número de devolución y aviso de revisión manual sin datos del pedido	2
	"Registro de devoluciones" con la estructura trigger, procesar y ejecutar, que se ejecuta sin errores desde la URL de prueba	2
	"Cierre semanal" con sus dos Schedule Trigger (viernes y lunes) y la zona horaria revisada	2
	Workspace ordenado: nombres y etiquetas de los workflows, nodos renombrados, nota en el canvas; los JSON se importan sin errores	2
Manipulando datos mediante nodos fundamentales	<i>Partes 2 y 3</i>	10
	Normalización con Edit Fields: <code>pedido_id</code> Number, <code>email</code> y <code>motivo</code> sin espacios y en minúsculas, fecha de recepción	2
	Expresiones que leen datos de otros nodos (<code>\$(...)</code>) y calculan <code>requiere_retiro</code> y el rango de fechas de la semana	2
	Summarize por motivo (cantidad y monto) y CSV para finanzas con los nombres de campo pedidos	2
	Filter de retiros; Merge por campo en común sin ítems duplicados ni choque de <code>id</code> o <code>estado</code> ; un solo XML con un <code><retiro></code> por devolución (Aggregate y nodo XML)	2
	Confirmación del transportista leída como JSON y separada con Split Out; umbral, monto mínimo y tablas en variables o en el nodo "Configuración"	2
Integrando bases de datos básicas	<i>Partes 2 y 3</i>	8
	Script ejecutado: <code>devoluciones</code> con clave foránea a <code>pedidos</code> , <code>log_devoluciones</code> y los seis pedidos de prueba	2
	<i>Get a row</i> del pedido por <code>id</code> , con <i>Always Output Data</i> para detectar el pedido inexistente	2
	<i>Create a row</i> en <code>devoluciones</code> y en <code>log_devoluciones</code> con los tipos correctos, incluidos <code>total_pedido</code> y <code>comuna</code> del pedido	2
	<i>Update a row</i> del pedido (<code>estado</code>) y de las devoluciones con retiro programado (<code>estado</code> y <code>fecha_retiro</code>)	2
De acuerdo con buenas prácticas de automatización	<i>Partes 2 y 4</i>	8
	Regla <code>requiere_aprobacion</code> con <code> </code> , que marca D4 por monto y D5 por ser mayorista	1
	Switch con tres rutas y salida de respaldo; pedido inexistente y correo que no coincide a revisión manual, con su detalle	2
	Toda ejecución deja una fila en <code>log_devoluciones</code> con ruta e <code>id_ejecucion</code> ; el error al registrar se maneja con la salida de error	2

Componente de la competencia	Qué se revisa	Puntos
	Plan de pruebas con 8 casos o más, resultado esperado y obtenido, ejecutados con la URL de prueba y con datos fijados, con evidencia en el historial	2
	Nota de depuración completa: síntoma, detección, causa, corrección y prueba	1
	Total	40

Nota (exigencia 60 %): si el puntaje p es mayor o igual que 24, $\text{nota} = 4,0 + 3 \times (p - 24) / 16$; si es menor, $\text{nota} = 1,0 + 3 \times p / 24$. Se redondea a un decimal. Por ejemplo: 20 puntos, 3,5; 24 puntos, 4,0; 32 puntos, 5,5; 40 puntos, 7,0.

Contenidos del plan que integra esta actividad

Parte de la actividad	Contenidos del plan
Parte 1, pasos 1 a 3: costo, retorno, por qué automatizar y beneficios del cambio	Conceptos fundamentales de workflow automation. Casos de uso empresariales. Beneficios y ROI de la automatización.
Parte 1, pasos 4 a 6: por qué n8n, diagrama de la solución y lista de nodos	Arquitectura de n8n. Componentes principales. Diferencias con otras plataformas de automatización. Ecosistema de nodos.
Parte 2, pasos 1 a 3: tablas, credencial, formulario y configuración	Integración con bases de datos. Supabase: configuración básica. Editor visual: nodos y conexiones. Primeros workflows. Configuración de nodos básicos. Variables de entorno.
Parte 2, pasos 4 a 7: normalizar, buscar el pedido, casos borde y armar la devolución	Set: modificación de datos. Manipulación de datos. Expresiones n8n: sintaxis y funciones. Panel de configuración. Manejo de casos edge. Operadores y expresiones complejas.
Parte 2, pasos 8 a 11: enrutar, registrar, dejar rastro y responder	Estructura trigger, procesar y ejecutar. If/Switch: lógica condicional. Routing de datos. Operaciones CRUD básicas. Manejo de datos relacionales Logs y trazabilidad.
Parte 3, pasos 1 a 6: resumen para finanzas y archivo de retiros	Filter: filtrado de datos. Summarize: agregación de datos. Merge: combinación de datos. Transformación de formatos. Manejo de arrays y objetos.
Parte 3, pasos 7 a 9: confirmación del transportista y actualización	Split: división de datos. Operaciones CRUD básicas. Logs y trazabilidad.
Parte 4: pruebas, depuración y orden del workspace	Testing y debugging inicial. Herramientas de debug en n8n. Mejores prácticas de testing. Workspace y navegación.

Después de entregar

- 1. Autoevaluación del módulo 2.** Al entregar, comparas cómo te veías al comenzar el módulo con cómo te ves hoy y revisas tu actividad final frente a la competencia del módulo.

2. **Retroalimentación del tutor/a.** Recibes tu puntaje por componente de la competencia, con una fortaleza, una mejora y un siguiente paso.
3. **Coevaluación por pares.** El tutor/a te asigna un compañero/a y te hace llegar por el LMS sus workflows y su informe: importas los workflows en tu instancia, los pruebas con tus propias tablas del insumo y le devuelves una retroalimentación con evidencia. Él o ella hace lo mismo con los tuyos.
4. **Portafolio.** Esta actividad es la evidencia de integración de tu portafolio «Automatizaciones para Mercado Austral». Ahí explicas cómo se conecta con lo que construiste durante el módulo.